UTN FRBA - Algoritmos y Estructuras de datos
Trabajo práctico grupal

Descripción general

Los drones militares pueden tener una ruta automatizada utilizando posicionamiento GPS. No obstante el enemigo puede utilizar bloqueadores de GPS para inhabilitar la señal satelital en una zona, dejando todos los dispositivos de navegación provisoriamente inoperativos. Para esos casos se debe realizar un sistema de guía de apoyo. El sistema, al no poder utilizar el GPS, utiliza una tabla de 200 x 200. En cada casillero puede haber una orden que posee la siguiente estructura:

```
struct Orden
{
     unsigned int espera  = 0;
     bool soltarGranada1 = false;
     bool soltarGranada2 = false;
     bool ataqueKamikaze = false;
     bool aterrizaje = false;
     bool despegue = false;
     int siguientex =0;
     int siguientey= 0;
}
```

Ud. y su equipo estará dedicado a la lógica offline de los drones. Para ello deberán escribir un programa en C++ que ejecute las órdenes indicadas. Para poder probar la lógica antes de ser instalada en los drones, el programa debe mostrar un menú por pantalla que muestre las siguientes opciones:

1. **Cargar archivo de ataque en memoria.**
2. **Mostrar ataque cargado.**
3. **Crear un archivo de ataque nuevo.**
4. **Corregir un registro del archivo.**
5. **Corregir un registro en memoria.**
6. **Guardar memoria en un archivo nuevo.**
7. **Visualizar un archivo de ataque en html.**

Operación 1: Cargar archivo de ataque en memoria

Al seleccionar esta operación, el usuario debe indicar ruta y nombre del archivo. El programa debe leer todos los registros del archivo y ubicarlos en las celdas correspondientes de la memoria del drone. Los registros que están en los archivos tienen la siguiente forma:

```
struct OrdenArchivo
{
     int x =0;
     int y =0;
```

```
    unsigned int espera  = 0;
    bool soltarGranada1 = false;
    bool soltarGranada2 = false;
    bool ataqueKamikaze = false;
    bool aterrizaje = false;
    bool despegue = false;
    int siguientex =0;
    int siguientey= 0;
}
```

Cuando se carga un archivo, deben tenerse en cuenta ciertas restricciones que, de no cumplirse, el programa debe dar aviso y no cargar el archivo.
Reglas de validación:
Una misma orden puede tener varias acciones en un mismo cuadrante, por lo tanto, se establece el orden de precedencia de las instrucciones es:

- Despegue
- Aterrizaje
- soltarGranada1
- soltarGranada2
- ataqueKamikaze
- Esperar
- Mover a la siguiente posición x y

Validaciones fundamentales:

- Los archivos deben tener una única instrucción de despegue.
- La instrucción de despegue no puede estar acompañada por ningún tipo de ataque.
- Debe existir una única instrucción de aterrizaje o una instrucción de ataqueKamikaze que finalice la operación del dron.
- No se puede pasar por las mismas coordenadas más de una vez
- La ruta debe estar completa, sin “huecos”, y se debe poder llegar desde la celda de despegue hasta el aterrizaje o ataque kamikaze por celdas aledañas.

Operación 2: Mostrar ataque
Esta opción debe mostrar en pantalla el ataque cargado en memoria. Cuidar de que no se exceda el tamaño de pantalla, si se debe pasar la pantalla a una nueva página, programar una manera de que el usuario presione una tecla para mostrar la siguiente página.

Operación 3: Crear un archivo de ataque nuevo
Esta operación permite al usuario crear un archivo de ataque nuevo. El usuario debe indicar ruta y nombre del archivo a crear. Luego, debe ir ingresando registro por registro.

Operación 4: Corregir un registro del archivo
El usuario debe indicar la posición x e y del registro a corregir de un archivo. El programa busca el registro y lo muestra por pantalla, el usuario puede ingresar nuevamente los datos y el registro en cuestión es reemplazado.

Operación 5: Corregir un registro en memoria

Igual a la operación 4 pero directamente en la tabla.

Operación 6: Guardar memoria en un archivo nuevo

Guarda en un nuevo archivo los registros que están en la tabla. El usuario puede ingresar ruta y nombre del archivo.

Operación 7: Visualizar un archivo de ataque

Genere una tabla html de 200x200 que muestre la tabla de ataques cargada en memoria. Para ello deberá escribir archivos de texto con las funciones que se muestran en el anexo. Además se explica brevemente el funcionamiento de los archivos html.

Condiciones de entrega

La fecha de entrega es el 4/11. La entrega debe ser con el programa compilado y funcionando.

Se debe entregar informe, con carátula que indique los legajos, nombres, apellidos y correos de los integrantes del grupo. En el informe se debe describir la solución con hipótesis tenidas en cuenta para la misma, diagrama de bloques de subprogramas y división de tareas en el equipo. Además se deben incluir capturas de pantalla con casos de prueba.

Se solicita por favor que todas herramientas que se hayan utilizado, especialmente aquellas relacionadas con IA, se referencien indicando:

- Nombre de la herramienta.
- Dirección URL.
- Por qué motivo y para qué se utilizó la herramienta.
- Respuesta otorgada por la misma.
- Cambios o adaptaciones realizadas sobre la respuesta otorgada por la herramienta.

La entrega debe realizarse en un repositorio GitHub que se coordinará con el docente.

Los grupos deben ser de hasta 5 integrantes sin excepción.

La aprobación del trabajo es condición necesaria para regularizar la materia.

Anexo 1: El lenguaje HTML

El lenguaje HTML es el utilizado para la confección de páginas web. Sus siglas representan las iniciales de *HyperText Markup Language* (lenguaje de marcas de hipertexto). HTML es una instancia, un caso de uso de un "XML".
Si bien, hoy en día, HTML se utiliza junto a otros lenguajes (como javascript, JQUERY desde el cliente y algún otro lenguaje que pueda ejecutarse desde el servidor, como PHP, ASP.NET, JSP, etc), la estructura de las páginas está especificada puramente en HTML.
Como todo lenguaje formal cumple un estándar normado por *World Wide Web Consortium* (W3C) o Consorcio WWW (https://www.w3.org/), organización dedicada a la estandarización de casi todas las tecnologías ligadas a la web, sobre todo en lo referente a su escritura e interpretación. Esta estandarización, permite que los navegadores web deban cumplir el estándar, permitiendo la existencia distintos productos para interpretar HTML.
En definitiva, un HTML es un archivo de texto (se puede crear con notepad), cuya característica principal es el uso de etiquetas para crear los elementos que componen la página. Dichos elementos no pueden estar en cualquier posición, por ejemplo, el elemento raíz debe ser una etiqueta <html> y todo el contenido de la página debe cerrarse cerrando dicha etiqueta con la etiqueta </html> (todos los elementos deben ser cerrados, aunque algunos pueden ser cerrados dentro de la misma etiqueta que se abren).

Etiquetas html básicas

- `<html>`: define el inicio del documento HTML, le indica al navegador que lo que viene a continuación debe ser interpretado como código HTML. Esto es así *de facto*, ya que en teoría lo que define el tipo de documento es el DOCTYPE, que significa la palabra justo tras DOCTYPE el tag de raíz.
- `<script>`: incrusta un script en una web, o llama a uno mediante `src="url del script"`. Se recomienda incluir el tipo MIME en el atributo `type`, en el caso de JavaScript `text/javascript`.
- `<head>`: define la cabecera del documento HTML; esta cabecera suele contener información sobre el documento que no se muestra directamente al usuario como, por ejemplo, el título de la ventana del navegador. Dentro de la cabecera `<head>` es posible encontrar:
- `<title>`: define el título de la página. Por lo general, el título aparece en la barra de título encima de la ventana.
- `<body>`: define el contenido principal o cuerpo del documento. Esta es la parte del documento html que se muestra en el navegador; dentro de esta etiqueta pueden definirse propiedades comunes a toda la página, como color de fondo y márgenes. Dentro del cuerpo `<body>` es posible encontrar numerosas etiquetas. A continuación se indican algunas a modo de ejemplo:
- `<h1>` a `<h6>`: encabezados o títulos del documento con diferente relevancia.
- `<table>`: define una tabla.
- `<tr>`: fila de una tabla.
- `<td>`: celda de una tabla (debe estar dentro de una fila).

Estas etiquetas se utilizan de forma anidada. Por ejemplo, una tabla de una sola fila que contiene una sola celda sería:

`<table><tr><td>Contenido de una celda</td></tr></table>`

- `<a>`: hipervínculo o enlace, dentro o fuera del sitio web. Debe definirse el parámetro de pasada por medio del atributo *href*. Por ejemplo: `<a href="http://www.example.com">Ejemplo</a>` se representa como ejemplo.
- `<img>`: imagen. Requiere del atributo *src*, que indica la ruta en la que se encuentra la imagen. Por ejemplo: `<img src="./imágenes/mifoto.jpg" />`. Es conveniente, por accesibilidad, poner un atributo `alt="texto alternativo"`.
- `<li><ol><ul>`: etiquetas para listas.
- `<b>`: texto en negrita (*Deprecado. Se reemplazó por la etiqueta* `<strong>`).
- `<i>`: texto en cursiva (*Deprecado. Se reemplazó por la etiqueta* `<em>`).
- `<s>`: texto tachado (*Deprecado. Se reemplazó por la etiqueta* `<del>`).
- `<u>`: Subrayado.

Nuevamente, la mayoría de etiquetas deben cerrarse como se abren, pero con una barra (/). Note, por ejemplo, la apertura y cierre del elemento
 del ejemplo de la figura 1.

Ejemplo HTML básico

El siguiente código muestra un ejemplo de un HTML básico que incluye en su cuerpo algunos elementos básicos listados en el punto anterior: un encabezado de tamaño 1, un párrafo, una tabla con dos filas y dos columnas y, finalmente, un vínculo a la página de la facultad. Por simplicidad, y para obviar por ahora los entities html, no se utilizan tildes dentro del código. Ud. puede crear este código, o uno similar, utilizado notepad u otro editor de texto (no procesador de texto como word o write, sino editor). Guarde el archivo con extensión .HTML para que el sistema operativo lo asocie directamente para ser abierto con el navegador predeterminado.

```
<html>
  <head>
    <title>Sintaxis y semantica de los lenguajes: Trabajo practico</title>
  </head>
  <body>
        <h1>Trabajo practico #1</h1>
    <p>Vamos a aplicar el conocimiento adquirido en nuestra materia a un lenguaje utilizado actualmente.</p>
        <table>
                <tr>
                <td>
                        Columna 1
                </td>
                <td>
                        Columna 2
                </td>
                </tr>
                <tr>
                <td>
                        1
                </td>
                <td>
                        2
                </td>
                </tr>
        </table>
        <br/> <!-- El br es un ejemplo de etiqueta que se abre y se cierra inmediatamente -->
        <a href="https://www.frba.utn.edu.ar/en/">Ir a pagina de la facu.</a>
  </body>
</html>
```

Figura 1 - Código HTML de la página de ejemplo

Al abrir el archivo descripto en la figura anterior se visualiza en el navegador web lo siguiente:

Trabajo practico #1

Vamos a aplicar el conocimiento adquirido en nuestra materia a un lenguaje utilizado actualmente.

Columna 1	Columna 2
1	2

Ir a pagina de la facu.

Figura 2 - Visualización del código de la figura 1

Anexo 2: Escritura de archivos de texto en C para generar un HTML

Como se explicó en el Anexo 1, un HTML es simplemente un archivo de texto. Por lo tanto, para generarlo desde un programa en C alcanza con usar las mismas funciones de manejo de archivos de texto: fopen para abrir o crear el archivo (esto es igual en archivos binarios, sólo cambia el modo de apertura), fprintf para escribir texto con formato dentro de él y fclose para cerrarlo, lo que garantiza que todo lo escrito se vuelque efectivamente a disco.

A continuación se muestra un ejemplo simplificado, aplicable a la Operación 7, que recorre una tabla en memoria y genera un archivo HTML con una tabla que la representa:

```
#include <stdio.h>

#define FILAS 200
#define COLUMNAS 200

void generarHTML(const char *nombreArchivo, int tabla[FILAS][COLUMNAS])
{
    FILE *archivo;
    int fila, columna;

    archivo = fopen(nombreArchivo, "w");
    if (archivo == NULL)
     {
        printf("No se pudo crear el archivo.\n");
        return;
     }

    fprintf(archivo, "<html>\n");
    fprintf(archivo, "<head><title>Ataque</title></head>\n");
    fprintf(archivo, "<body>\n");
    fprintf(archivo, "<table border=\"1\">\n");

    for (fila = 0; fila < FILAS; fila++)
     {
        fprintf(archivo, "<tr>\n");
        for (columna = 0; columna < COLUMNAS; columna++)
        {
            fprintf(archivo, "<td>%d</td>", tabla[fila][columna]);
        }
        fprintf(archivo, "</tr>\n");
     }

    fprintf(archivo, "</table>\n");
    fprintf(archivo, "</body>\n");
    fprintf(archivo, "</html>\n");

    fclose(archivo);
}
```

Cada llamada a fprintf agrega al archivo el fragmento de HTML correspondiente (las etiquetas de apertura y cierre del documento, del cuerpo y de la tabla, y una celda por cada casillero de la matriz).

Se muestra un ejemplo de visualización. Considere que este ejemplo tiene estilos CSS, está “adornado”. Puede ver el código fuente de este html como adjunto en el mismo zip de donde obtuvo este enunciado. El programa que Ud. y su equipo confeccionen puede generar ejemplos mucho más rústicos:

